🚀 Guía: Desplegar AgentOS en Vercel desde Local

Para pasar este proyecto (tu nuevo Sistema Operativo de Agentes) a tu entorno local y desplegarlo en Vercel, usaremos **Vite** (un empaquetador moderno y ultrarrápido para React) y **Tailwind CSS** para los estilos.

Sigue estos pasos en tu terminal:

Paso 1: Crear el proyecto local con Vite

Abre tu terminal y ejecuta el siguiente comando para crear un proyecto React con Vite. Cuando te pregunte, elige `React` y luego `JavaScript` (o SWC):

```
npm create vite@latest agentos -- --template react
cd agentos
npm install
```

Paso 2: Instalar las dependencias (Iconos y Tailwind)

El proyecto utiliza iconos de `lucide-react` y clases de Tailwind CSS. Instálalos con estos comandos:

```
# Instalar los iconos
npm install lucide-react

# Instalar Tailwind CSS y sus dependencias
npm install -D tailwindcss postcss autoprefixer

# Generar los archivos de configuración de Tailwind
npx tailwindcss init -p
```

Paso 3: Configurar Tailwind CSS

Abre el archivo recién creado `tailwind.config.js` en la raíz de tu proyecto y reemplaza su contenido por esto para que Tailwind lea tus componentes de React:

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
```

```
  plugins: [],
}
```

Luego, abre el archivo `src/index.css` (borra todo lo que tenga) y pega las directivas principales de Tailwind:

```
@tailwind base;
@tailwind components;
@tailwind utilities;

/* Añade esto para asegurar que el body ocupe toda la pantalla y tenga el fondo c
html, body, #root {
  height: 100%;
  width: 100%;
  margin: 0;
  background-color: #1e1e1e;
}
```

Paso 4: Integrar el código de la App

1. Ve a la carpeta `src/` .
2. Abre el archivo `App.jsx` .
3. **Borra todo el contenido** de ese archivo.
4. **Pega todo el código** del componente de React (el archivo principal del POM) que hemos estado creando.
5. *(Opcional)* Si ves algún problema con React no importado, asegúrate de que la primera línea de tu `App.jsx` sea `import React, { useState, useEffect } from 'react';` .

Prueba que funciona en local ejecutando:

```
npm run dev
```

(Abre `http://localhost:5173` en tu navegador, debería verse el ecosistema de AgentOS funcionando perfectamente).

Paso 5: Subir el proyecto a GitHub

Vercel funciona mejor cuando está conectado a un repositorio de GitHub (así se actualiza solo cada vez que haces un `push`).

```
git init
git add .
```

```
git commit -m "🚀 Primer commit: AgentOS configurado"
```

1. Ve a GitHub y crea un nuevo repositorio vacío llamado `agentos`.
2. Copia los comandos que te da GitHub para empujar un repositorio existente. Serán algo así:

```
git remote add origin [https://github.com/tu-usuario/agentos.git](https://github.
git branch -M main
git push -u origin main
```

Paso 6: Desplegar en Vercel 🚀

¡El paso más fácil!

1. Ve a Vercel e inicia sesión con tu cuenta de GitHub.
2. Haz clic en el botón **"Add New..."** y selecciona **"Project"**.
3. Verás una lista con tus repositorios de GitHub. Busca `agentos` y haz clic en **"Import"**.
4. Vercel detectará automáticamente que es un proyecto de Vite/React. No necesitas cambiar ninguna configuración (Build Command: `npm run build`, Output Directory: `dist`).
5. Haz clic en **"Deploy"**.

En menos de un minuto, Vercel compilará tu aplicación y te dará una URL pública en vivo (ejemplo: `agentos.vercel.app` o tu dominio personalizado `agentos.app`).

🎉 **¡Listo!** A partir de ahora, cualquier cambio que hagas en tu código local, solo tienes que hacer `git push` y Vercel lo actualizará automáticamente en la web.